Módulo 02: Deploy Containerized Web Applications to Azure App Service

PaaS de contenedores en Linux, autenticación Zero-Trust con Identidad Administrada y persistencia en Azure App Service

Alineación Curricular Oficial Microsoft Learn: Este módulo práctico pertenece al **Curso AI-200T00-A: Desarrollo de soluciones en la nube de IA en Azure** (5 días / 120 hrs) y cubre las competencias del **Dominio 1: Desarrollo de soluciones en contenedores en Azure (20% – 25%)** de la certificación oficial **Microsoft Certified: Azure AI Cloud Developer Associate (Examen AI-200)**. Consulte la Guía de Estudio Oficial AI-200 en Microsoft Learn.

PRERREQUISITOS UNIVERSALES DEL ENTORNO

- **Suscripción de Azure:** Requiere plan con soporte de App Service Linux dedicado (ej. `P0v3` o `B1`).
- **Azure CLI:** Versión 2.50+ instalada localmente o en Azure Cloud Shell (`az --version`).
- **Python 3.10+:** Para ejecutar scripts de despliegue automatizado de infraestructura (`azdeploy.py`).
- **Permisos RBAC:** Roles `Contributor` y `User Access Administrator` u `Owner` para asignación de roles.
- **Sin Docker local:** No se requiere Docker Desktop; la imagen se compila en ACR Tasks y se corre en PaaS.

CONSIDERACIONES Y AVISOS CRÍTICOS DE AZURE

- **Sistema Operativo Linux:** Los contenedores personalizados requieren un App Service Plan con el flag `--is-linux`.
- **Unicidad global de DNS:** El nombre de la Web App debe ser único mundialmente (`<app>.azurewebsites.net`).
- **Mapeo de puertos en contenedor:** Por defecto App Service asume el puerto 80; si difiere, requiere `WEBSITES_PORT`.
- **Almacenamiento en contenedor:** El sistema es efímero salvo que se active `WEBSITES_ENABLE_APP_SERVICE_STORAGE=true`.

Estructura del Proyecto Fuente: Paquete oficial descargado en `CODE/2. app-svc-container-python/` que contiene la API Flask de procesamiento documental (`api/main.py`), el archivo de construcción OCI (`api/Dockerfile`), requisitos (`api/requirements.txt`), el script de despliegue (`azdeploy.py`) y el archivo de prueba (`document.txt`).

Fundamentos y Jerarquía de Azure CLI para Azure App Service

Estructura jerárquica de comandos: El ciclo de vida de Web App for Containers se administra mediante dos grupos: `az appservice plan` (asigna cómputo físico, CPU y memoria) y `az webapp` (gestiona la configuración lógica, identidades, contenedores y variables).

Parámetro / Flag	Propósito Técnico en App Service	Ejemplo Práctico de Uso
`--plan`	Nombre o ID del App Service Plan que provee el cómputo subyacente.	`--plan plan-docprocessor-a1b2c3d4`
`--deployment-container-image-name` (`-i`)	Especifica la imagen OCI y etiqueta alojada en ACR para el contenedor.	`--deployment-container-image-name acr.azurecr.io/docprocessor:v1`
`--assign-identity` (o `identity assign`)	Habilita una identidad administrada asignada por el sistema en Entra ID.	`az webapp identity assign --name $APP_NAME ...`
`--role AcrPull`	Rol RBAC mínimo para autenticar y extraer capas de contenedor desde ACR.	`--role "AcrPull" --scope $ACR_ID`
`--acr-use-identity` / `--docker-registry-server-url`	Configura al motor Kudu para que use el token MSI en lugar de contraseñas.	`az webapp config container set --acr-use-identity true ...`
`--settings`	Inyecta variables de entorno en el contenedor (Application Settings).	`--settings WEBSITES_ENABLE_APP_SERVICE_STORAGE=true`
`--docker-container-logging`	Habilita el enrutamiento de stdout/stderr hacia el sistema de archivos o blob.	`az webapp log config --docker-container-logging filesystem`

Fase 1: Preparación del Entorno, Identidad y Proveedores de Recursos

`az login & az provider register (Microsoft.Web)`

Identidad & API

Inicie sesión en Azure y registre los proveedores de recursos para Web Apps y registros de contenedores:

```
# 1. Autenticación interactiva en la suscripción
az login

# 2. Verificar la suscripción activa actual
az account show --output table

# 3. Registrar los Resource Providers obligatorios: Microsoft.Web y Microsoft.ContainerRegistry
az provider register --namespace Microsoft.Web
az provider register --namespace Microsoft.ContainerRegistry

# 4. Verificar el estado del proveedor de App Service (debe retornar "Registered")
az provider show `
  --namespace Microsoft.Web `
  --query "registrationState" `
  -o tsv
```

¿Por qué registrar Microsoft.Web?: Es el proveedor ARM responsable del aprovisionamiento de App Service Plans, Web Apps for Containers, certificados SSL administrados y front-ends de balanceo de carga en Azure.

INSPECCIÓN Y GESTIÓN DE PROVEEDORES EN AZURE PORTAL (GUI)

Navegación: Busque **Subscriptions** en la barra superior -> Seleccione su suscripción -> En el menú izquierdo bajo **Settings**, haga clic en **Resource providers** -> Filtre por `Microsoft.Web` y verifique que figure en estado **Registered** (si está en *NotRegistered*, selecciónelo y pulse **Register**).

Aprovisionamiento Automatizado con Python y App Service Plan

Ejecución de azdeploy.py, compilación serverless de imagen y creación del plan Linux P0v3

ANATOMÍA Y FUNCIONAMIENTO DE AZDEPLOY.PY

El script `azdeploy.py` actúa como un orquestador idempotente: recupera el `user_object_id` de Entra ID, deriva un sufijo criptográfico SHA-1 de 8 caracteres para asegurar la unicidad mundial de DNS (`acr<hash>`, `plan-docprocessor-<hash>`, `app-docprocessor-<hash>`), crea el registro ACR y el plan de hospedaje Linux.

Paso 2.1: Crear Azure Container Registry y Compilar la Imagen del Contenedor

`python azdeploy.py (Opción 1: ACR & Build)` — ACR Tasks Serverless

Configure las variables de Resource Group y región en `azdeploy.py` (líneas 5 y 6) y ejecute la opción 1 del menú interactivo:

```
# 1. Ejecutar el script interactivo desde CODE/2. app-svc-container-python/
python azdeploy.py

# En el menú presentado, seleccione la Opción 1:
# 1. Create Azure Container Registry and build container image
# Salida generada:
Checking/creating resource group 'rg-docprocessor-lab'...
Creating Azure Container Registry 'acra1b2c3d4'...
ACR created: acra1b2c3d4. Login server: acra1b2c3d4.azurecr.io
Building and pushing container image to ACR...
Image built and pushed: acra1b2c3d4.azurecr.io/docprocessor:v1
```

Compilación serverless en la nube: La opción 1 invoca `az acr build` empaquetando `api/Dockerfile` y `api/main.py` en un tarball enviado por TLS a ACR Tasks, entregando la imagen `docprocessor:v1` lista sin depender de Docker local.

Paso 2.2: Crear el App Service Plan en Linux (P0v3) y Cargar Variables

`python azdeploy.py (Opción 2: App Service Plan)` — Cómputo Linux PaaS

Seleccione la opción 2 para aprovisionar el plan de proceso dedicado y exportar las variables de entorno:

```
# 2. En el menú de azdeploy.py, seleccione la Opción 2:
# 2. Create App Service Plan
Creating App Service Plan 'plan-docprocessor-a1b2c3d4'...
App Service Plan created: plan-docprocessor-a1b2c3d4 (SKU: P0v3)
Environment variables saved to: .env and .env.ps1

# 3. Cargar las variables en su sesión de terminal activa:
# En Bash / macOS / Linux:
source .env

# En Windows PowerShell:
. .\.env.ps1

# 4. Verificar las variables exportadas:
az appservice plan show -g $RESOURCE_GROUP -n $APP_PLAN --query '{Name:name, SKU:sku.name, OS:reserved}' -o table
```

NIVELES DE SKU DEL APP SERVICE PLAN

- **Basic (B1):** 1 núcleo vCPU, 1.75 GB RAM. Nivel mínimo dedicado para desarrollo y pruebas de contenedores Linux.
- **Premium v3 (P0v3 / P1v3):** Diseñado para producción de IA con procesadores rápidos, escalado automático hasta 30 réplicas y soporte de VNet Integration.
- **Flag `--is-linux` (`reserved: true`):** Asigna nodos Linux dedicados para ejecutar contenedores OCI de forma nativa.

INSPECCIÓN EN AZURE PORTAL (GUI)

1. **Grupo de Recursos:** Ingrese a **Resource groups** -> Seleccione `rg-docprocessor-lab`.
2. **Container Registry:** Haga clic en `acra1b2c3d4` -> En **Repositories** verifique el tag `docprocessor:v1`.
3. **App Service Plan:** Haga clic en `plan-docprocessor-...` -> En el panel general verifique: **Operating System: Linux** y **Pricing tier: P0v3**.

Creación de Web App for Containers y Detección de Puertos

Aprovisionamiento del recurso webapp, proxy reverso PaaS y configuración de arranque del contenedor

Paso 3.1: Crear la Web App apuntando a la Imagen de Azure Container Registry

`az webapp create (--deployment-container-image-name)`

PaaS Container Instance

Cree el recurso Web App en el plan de hospedaje Linux indicando la imagen compilada previamente en ACR:

```
# Comando multilínea compatible con PowerShell ( ` ) y Bash ( \ )
az webapp create `
  --resource-group $RESOURCE_GROUP `
  --plan $APP_PLAN `
  --name $APP_NAME `
  --deployment-container-image-name "$ACR_NAME.azurecr.io/docprocessor:v1"

# 2. Consultar el estado de aprovisionamiento de la Web App
az webapp show `
  --resource-group $RESOURCE_GROUP `
  --name $APP_NAME `
  --query '{State:state, Hostname:defaultHostName, LinuxFx:siteConfig.linuxFxVersion}' `
  -o table
```

Propiedad `linuxFxVersion`: Define el runtime en formato `DOCKER|<login-server>/<image>:<tag>`. Indica al subsistema Kudu de App Service qué contenedor OCI debe ejecutar en el nodo host.

MAPEO DE PUERTOS Y ENRUTAMIENTO INTERNO

- **Detección automática:** App Service analiza el manifiesto de la imagen y detecta la cláusula `EXPOSE 80` del `Dockerfile`.
- **Proxy inverso frontal:** El balanceador PaaS recibe solicitudes públicas en los puertos 80/443 (HTTPS) y las redirige internamente al puerto en el que escucha Gunicorn (puerto 80).
- **Variable `WEBSITES_PORT`:** Si la aplicación escucha en un puerto personalizado (ej. 5000 o 8080), es obligatorio definir `WEBSITES_PORT=5000` en Application Settings.
- **Enlace a `0.0.0.0`:** En contenedores Docker, la aplicación debe enlazar a `0.0.0.0` (no a `127.0.0.1`), de lo contrario el proxy de App Service no podrá alcanzar el servicio.

TIMEOUT DE INICIO DEL CONTENEDOR

- **Límite predeterminado:** App Service espera hasta **230 segundos** a que el contenedor responda a las sondas HTTP iniciales antes de abortar el arranque.
- **Modelos pesados de IA:** Si la aplicación descarga embeddings o inicializa pesos en memoria durante el arranque, este tiempo puede ser insuficiente.
- **Variable de ampliación:** Se debe configurar `WEBSITES_CONTAINER_START_TIME_LIMIT=600` (configurable hasta un máximo de 1800 segundos / 30 minutos).
- **Diagnóstico de fallos:** Si el contenedor se reinicia en bucle (crash loop), revise los logs del motor Docker en Kudu.

Paso a Paso en Azure Portal (GUI) para Crear Web App for Containers

1. **Búsqueda y Creación:** En la barra superior busque **App Services** -> Pulse **+ Create** -> **Web App**.
2. **Pestaña Basics:**
   - **Resource Group:** Seleccione `rg-docprocessor-lab`.
   - **Name:** Ingrese el nombre único global (ej. `app-docprocessor-a1b2c3d4`). Debe generar el FQDN verde de validación.
   - **Publish:** Seleccione **Container** (no Code).
   - **Operating System:** Seleccione **Linux**.
   - **Region & Plan:** Seleccione la misma región que el ACR y el App Service Plan existente `plan-docprocessor-...`.
3. **Pestaña Container:** Marque **Single Container** -> Image Source: **Azure Container Registry** -> Registry: `acra1b2c3d4` -> Image: `docprocessor` -> Tag: `v1`.
4. **Finalización:** Haga clic en **Review + create** y confirme la creación del recurso.

Seguridad Zero-Trust con Identidad Administrada y Rol AcrPull

Eliminación de contraseñas estáticas mediante System-Assigned Managed Identity y asignación RBAC en Entra ID

Principio de Mínimos Privilegios (Zero-Trust): Nunca habilite la cuenta `--admin-enabled true` en registros de producción. Utilice identidades administradas asignadas por el sistema (MSI): no requieren almacenar credenciales en App Settings, rotan automáticamente cada 24 horas y restringen el acceso exclusivamente a extracción con el rol `AcrPull`.

Paso 4.1: Habilitar la Identidad Administrada Asignada por el Sistema en la Web App

`az webapp identity assign (--query principalId)` — Microsoft Entra ID

Active la identidad administrada en la Web App para que Azure registre un Service Principal en Entra ID:

```
# 1. Habilitar la identidad asignada por el sistema y capturar el principalId
PRINCIPAL_ID=$(az webapp identity assign `
  --resource-group $RESOURCE_GROUP `
  --name $APP_NAME `
  --query "principalId" `
  -o tsv)

# 2. Verificar el Object (Principal) ID asignado en Entra ID
echo "Managed Identity Principal ID: $PRINCIPAL_ID"
```

Ciclo de vida de System-Assigned Identity: La identidad comparte el ciclo de vida del recurso Web App. Si la aplicación se elimina, Azure elimina automáticamente el Service Principal en Entra ID y todas sus asignaciones de roles.

Paso 4.2: Asignar el Rol RBAC AcrPull en el Ámbito del Azure Container Registry

`az role assignment create (--role AcrPull --scope $ACR_ID)` — RBAC Authorization

Obtenga el identificador de recurso de ACR y asigne el rol de extracción mínima a la Web App:

```
# 1. Obtener el Resource ID completo del registro ACR
ACR_ID=$(az acr show `
  --resource-group $RESOURCE_GROUP `
  --name $ACR_NAME `
  --query "id" `
  -o tsv)

# 2. Otorgar permiso de extracción (AcrPull) a la identidad de la Web App
az role assignment create `
  --assignee $PRINCIPAL_ID `
  --role "AcrPull" `
  --scope $ACR_ID

# 3. Comprobar la asignación de rol activa
az role assignment list --assignee $PRINCIPAL_ID --output table
```

Definición del rol AcrPull: Autoriza la acción `Microsoft.ContainerRegistry/registries/pull/read`. Impide que un atacante que comprometa la Web App pueda modificar imágenes o publicar artefactos maliciosos (requeriría `AcrPush`).

Paso a Paso en Azure Portal (GUI) para Identidad y Control de Acceso (IAM)

1. **Habilitar Managed Identity en la Web App:**
   - Navegue a su **App Service** (`app-docprocessor-...`) -> En el menú lateral izquierdo bajo **Settings**, haga clic en **Identity**.
   - En la pestaña **System assigned**, cambie el interruptor **Status** a **On** -> Pulse **Save** y confirme con **Yes**.
   - Verifique que aparezca el **Object (principal) ID** generado en pantalla.
2. **Otorgar Rol AcrPull en Azure Container Registry:**
   - Navegue a su **Container registry** (`acra1b2c3d4`) -> Haga clic en **Access control (IAM)** en el menú izquierdo.
   - Pulse **+ Add** -> **Add role assignment** -> En la pestaña **Role** busque y seleccione **AcrPull** -> Siguiente.
   - En la pestaña **Members**, seleccione **Managed identity** -> Pulse **+ Select members**.
   - En el panel flotante elija **Managed identity: App Service**, seleccione su Web App y pulse **Select** -> **Review + assign**.

Extracción sin Secretos y Almacenamiento Persistente

Activación de acrUseManagedIdentityCreds, montaje de /home y configuración en Application Settings

Paso 5.1: Configurar a Kudu para Autenticar contra ACR usando la Identidad Administrada

az webapp config container set & az resource update — Passwordless Pull

Configure el orquestador Kudu de App Service para que solicite tokens a Entra ID al descargar la imagen:

```
# 1. Configurar la URL del servidor de registro y la imagen
az webapp config container set `
  --resource-group $RESOURCE_GROUP `
  --name $APP_NAME `
  --docker-custom-image-name "$ACR_NAME.azurecr.io/docprocessor:v1" `
  --docker-registry-server-url "https://$ACR_NAME.azurecr.io"

# 2. Habilitar la propiedad acrUseManagedIdentityCreds en el plano de control ARM
az resource update `
  --ids "/subscriptions/$(az account show --query id -o tsv)/resourceGroups/$RESOURCE_GROUP/providers/Microsoft.Web/sites/$APP_NAME/config/web"
`
  --set properties.acrUseManagedIdentityCreds=true
```

Eliminación de secretos: Con esta directiva, no es necesario suministrar `DOCKER_REGISTRY_SERVER_USERNAME` ni `DOCKER_REGISTRY_SERVER_PASSWORD`. Kudu utiliza el token OAuth 2.0 emitido por el servicio IMDS local de la máquina virtual host.

Paso 5.2: Habilitar Almacenamiento Persistente en /home y Ajustes de Producción

az webapp config appsettings set (WEBSITES_ENABLE_APP_SERVICE_STORAGE) — Persistent Storage

Inyecte las variables de configuración de la API y active el almacenamiento persistente respaldado por Azure Storage:

```
# Configurar persistencia, timeouts y variables operativas de la API Flask
az webapp config appsettings set `
  --resource-group $RESOURCE_GROUP `
  --name $APP_NAME `
  --settings `
    WEBSITES_ENABLE_APP_SERVICE_STORAGE=true `
    STORAGE_PATH=/home/processed `
    ENVIRONMENT=production `
    LOG_LEVEL=INFO `
    MAX_DOCUMENT_SIZE_MB=50 `
    PROCESSING_TIMEOUT_SECONDS=30 `
    WEBSITES_CONTAINER_START_TIME_LIMIT=600
```

ARQUITECTURA DEL VOLUMEN /HOME PERSISTENTE

- **Comportamiento efímero estándar:** En contenedores Linux sin almacenamiento de App Service, los archivos creados en el contenedor se escriben en memoria o disco efímero y se destruyen al reiniciar la app.
- **Montaje CIFS/NFS:** Al activar `WEBSITES_ENABLE_APP_SERVICE_STORAGE=true`, la infraestructura de Azure monta un volumen compartido cifrado en `/home`.
- **Resiliencia de documentos:** Todos los análisis JSON guardados por la API en `/home/processed` sobreviven a reinicios, despliegues y conmutaciones por error.

INSPECCIÓN EN AZURE PORTAL (GUI)

1. Variables de Aplicación: En la Web App -> Menú izquierdo **Settings** -> **Configuration** -> En **Application settings** confirme que `WEBSITES_ENABLE_APP_SERVICE_STORAGE` esté en `true`.

2. Deployment Center: En **Deployment** -> **Deployment Center** -> Pestaña **Settings** -> Verifique que el Registry apunte a su ACR y **Authentication** figure como **Managed Identity**.

Pruebas Funcionales de API, Streaming de Logs y Limpieza

Verificación de endpoints con curl, telemetría en tiempo real con az webapp log tail y eliminación de recursos

Paso 6.1: Validación de Endpoints de Salud, Ingesta y Persistencia

curl (Health, Process & Documents Endpoints)

API Verification

Ejecute solicitudes HTTP contra la Web App pública para validar la respuesta del servidor Gunicorn/Flask:

```
# 1. Comprobación del probe de salud (Liveness Probe)
curl -s https://$APP_NAME.azurewebsites.net/health
{"status":"healthy"}

# 2. Información del servicio y entorno activo
curl -s https://$APP_NAME.azurewebsites.net/
{"environment":"production","service":"Document Processing API","status":"running","version":"1.0.0"}

# 3. Procesar un documento enviando contenido de texto (POST /process)
curl -s -X POST https://$APP_NAME.azurewebsites.net/process `
  -H "Content-Type: application/json" `
  -d '{"content": "Azure App Service executes containerized microservices with Managed Identity.", "filename": "doc1.txt"}'

# 4. Listar documentos guardados en el almacenamiento persistente /home/processed
curl -s https://$APP_NAME.azurewebsites.net/documents
{"count":1,"documents":[{"created_at":"...","document_id":"d1e2f3...","size_bytes":842}]}
```

Paso 6.2: Habilitación de Diagnósticos y Transmisión de Logs en Tiempo Real

az webapp log config & az webapp log tail

Observabilidad

Active el registro de contenedores en el sistema de archivos y transmita la consola en tiempo real:

```
# 1. Habilitar la captura de logs de contenedor en el sistema de archivos de App Service
az webapp log config `
  --resource-group $RESOURCE_GROUP `
  --name $APP_NAME `
  --docker-container-logging filesystem

# 2. Iniciar la transmisión en vivo de logs (Live Log Streaming)
az webapp log tail --resource-group $RESOURCE_GROUP --name $APP_NAME

# Salida típica observada en la consola:
2026-10-15T22:00:10Z [INFO] Pulling image: acra1b2c3d4.azurecr.io/docprocessor:v1
2026-10-15T22:00:14Z [INFO] Successfully pulled image from registry
2026-10-15T22:00:15Z [INFO] Starting container for site: app-docprocessor-a1b2c3d4
2026-10-15T22:00:16Z [INFO] [Gunicorn] Starting gunicorn 22.0.0
2026-10-15T22:00:16Z [INFO] [Gunicorn] Listening at: http://0.0.0.0:80 (PID 1)
2026-10-15T22:00:17Z [INFO] [Flask] Starting Document Processing API in production environment
2026-10-15T22:00:18Z [INFO] Container app-docprocessor-a1b2c3d4 for site app-docprocessor-a1b2c3d4 initialized successfully and is ready to ser
ve requests.
```

INSPECCIÓN EN PORTAL (LOG STREAM & SSH)

- **Log Stream:** En la Web App -> **Monitoring** -> **Log stream**: Visualización web continua de los eventos de inicio y peticiones entrantes.
- **Consola WebSSH (Kudu):** En **Development Tools** -> **SSH**: Terminal interactiva en el contenedor. Ejecute `ls -la /home/processed` para verificar los JSON persistidos.

FASE DE LIMPIEZA DE RECURSOS

- **Comando asíncrono no bloqueante:**

`az group delete -n $RESOURCE_GROUP --no-wait --yes`

- **Por qué es crítico:** Los App Service Plans con SKU `P0v3` facturan cómputo dedicado por hora independientemente de si reciben tráfico.

Diagrama de Arquitectura Lógica de la Solución

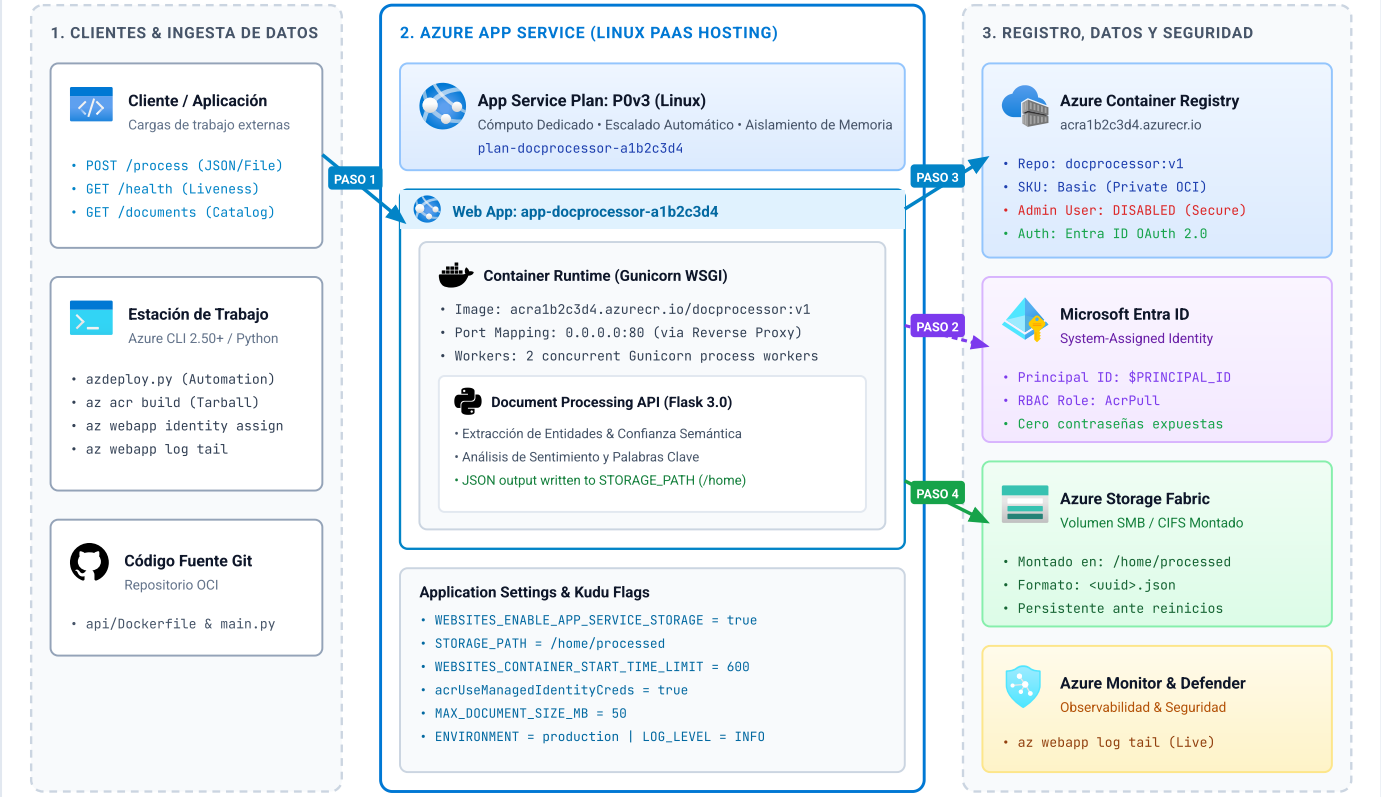
Caso de uso empresarial de procesamiento inteligente de documentos, desacoplamiento PaaS y autenticación Zero-Trust

CASO DE USO REAL: INGESTA Y ANÁLISIS SEGURO DE DOCUMENTOS LEGALES Y FINANCIEROS

Problemática Empresarial: Una entidad corporativa procesa diariamente contratos confidenciales y reportes financieros mediante modelos de IA. Anteriormente utilizaban servidores locales con credenciales maestras compartidas en texto plano, sin persistencia ante fallos y con caídas frecuentes durante picos de carga.
Solución de Arquitectura en Azure: Migración a un microservicio en contenedor alojado en **Azure App Service en Linux**. La imagen se almacena en **Azure Container Registry (ACR)** y el contenedor se autentica de forma segura mediante **System-Assigned Managed Identity** con el rol `AcrPull`, sin exponer contraseñas. Los resultados analíticos se almacenan en el volumen persistente `/home/processed` respaldado por Azure Storage.

BLUEPRINT DE ARQUITECTURA LÓGICA (DRAW.IO CANVAS ESTILO VECTORIAL)

Zero-Trust & Persistent PaaS



Secuencia Lógica: (1) Ingesta: Peticiones HTTPS entrantes balanceadas al contenedor Flask en el puerto 80. **(2) Autenticación Zero-Trust:** App Service presenta el token de Entra ID vía Managed Identity sin credenciales fijas. **(3) Descarga OCI:** Kudu extrae las capas de `docprocessor:v1` desde ACR con permisos `AcrPull`. **(4) Persistencia Segura:** Los análisis procesados se escriben en `/home/processed` respaldado por Azure Storage resiliente.

Diagrama de Arquitectura Física y Red de App Service

Topología de red perimetral, plano de datos aislado, servicio IMDS y montaje CIFS/SMB cifrado

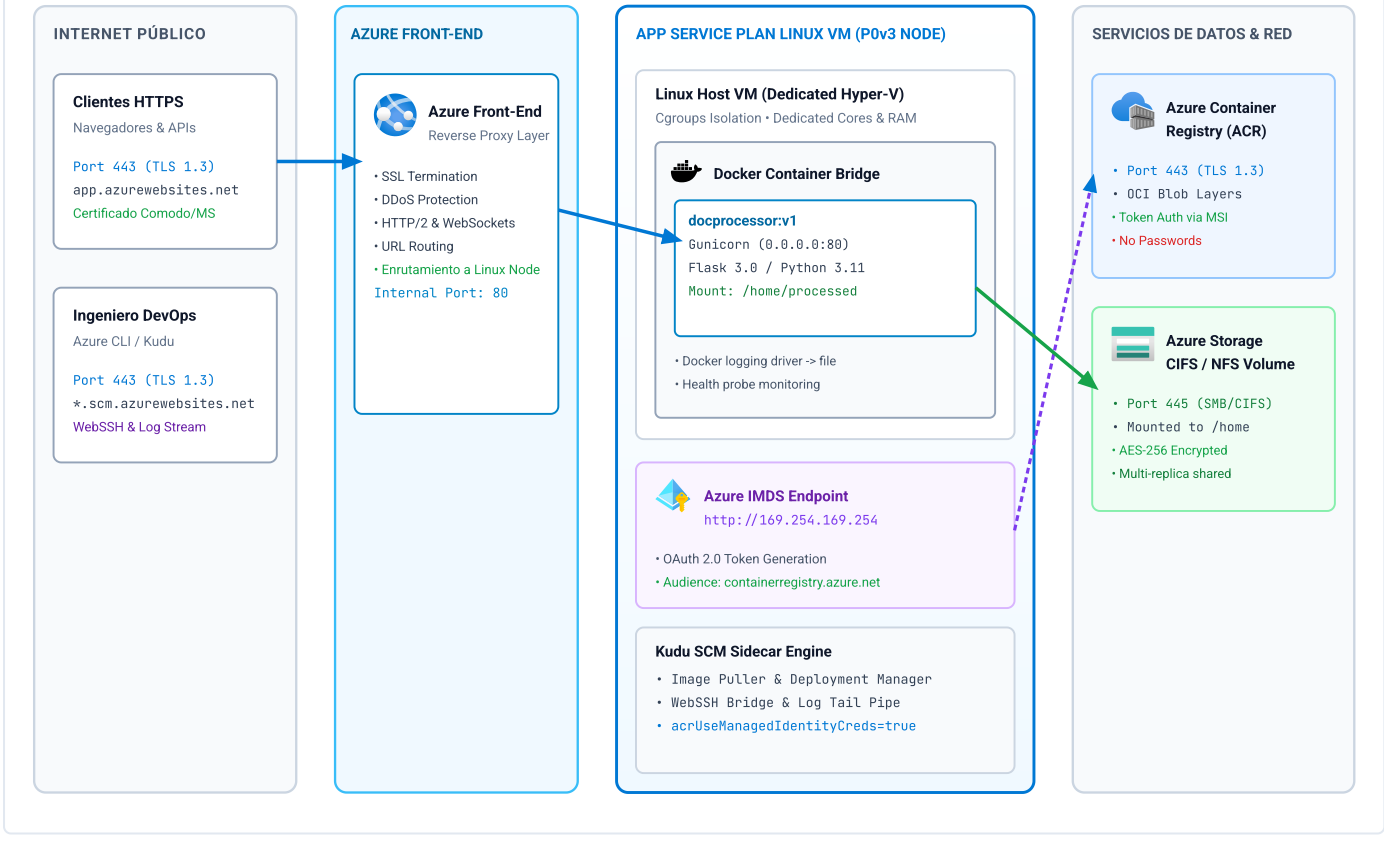
TOPOLOGÍA DE RED Y FRONTERAS DE AISLAMIENTO FÍSICO

Aislamiento de Cómputo: El App Service Plan Linux aprovisiona instancias de máquinas virtuales dedicadas protegidas por Azure Hyper-V y cgroups. El tráfico de producción ingress en el puerto 443 es terminado en el balanceador frontal PaaS de Azure y reenviado por un canal local al contenedor Docker en `0.0.0.0:80`.

Autenticación Criptográfica IMDS: La identidad administrada consulta internamente al Azure Instance Metadata Service (`http://169.254.169.254/metadata/identity/oauth2/token`), una dirección local no enrutable a Internet. Los tokens obtenidos son enviados por TLS 1.3 hacia el plano de datos de Azure Container Registry.

TOPOLOGÍA DE RED Y PLANO FÍSICO (DRAW.IO CANVAS ESTILO VECTORIAL)

TLS 1.3 • IMDS 169.254.169.254 • SMB Port 445



Aspectos Clave de Seguridad de Red: (1) Terminación TLS: El balanceador PaaS frontal de Azure gestiona certificados SSL y mitiga ataques de capa 7 antes de alcanzar el nodo Linux. **(2) Cero Tráfico de Credenciales:** El daemon IMDS entrega tokens en memoria sin exponer secretos en el canal de red público. **(3) Montaje de Red Cifrado:** El almacenamiento en `/home` se comunica por canal CIFS interno cifrado con AES-256 gestionado por Microsoft.

Disección Exhaustiva del Código Fuente y Configuración

Análisis técnico de azdeploy.py, contenedor OCI en Dockerfile y contratos REST en main.py

1. Script de Despliegue de Infraestructura: azdeploy.py

Hash Criptográfico y Unicidad Global: En las líneas 121-127, la función `_derived_names(user_object_id)` calcula un resumen SHA-1 truncado a 8 caracteres basado en el ID de Entra ID del usuario. Esto garantiza que `acr<hash>`, `plan-docprocessor-<hash>` y `app-docprocessor-<hash>` cumplan con la restricción de FQDNs únicos mundiales en Azure sin colisiones.

Resolución Multiplataforma de Binarios: En `_resolve_exe()` (líneas 27-37), utiliza `shutil.which()` para resolver automáticamente `az.cmd` en Windows o `az` en entornos Unix/macOS, almacenando el ejecutable en caché para optimizar llamadas repetitivas.

Generación Dual de Entorno: En `write_env_files()` (líneas 88-119), genera en paralelo `.env` (usando `export KEY="VAL"` con escapado Bash) y `.env.ps1` (usando `$env:KEY = "VAL"` con acentos graves de PowerShell) en codificación UTF-8 pura con saltos de línea LF.

2. Especificación del Contenedor OCI: api/Dockerfile

Imagen Base Hardened (Línea 1): `FROM python:3.11-slim` reduce drásticamente el tamaño del artefacto (de ~1 GB a menos de 150 MB) y elimina librerías de compilación innecesarias, mitigando vectores de vulnerabilidad (CVEs).

Higiene de Capas (Líneas 6-7): Instala dependencias copiando únicamente `requirements.txt` con `pip install --no-cache-dir`. Esto previene que los archivos temporales de pip se incorporen en las capas inmutables de la imagen Docker.

Directorio de Persistencia (Línea 13): `RUN mkdir -p /home/processed` asegura que el punto de montaje exista antes de que App Service enlace el volumen compartido CIFS.

Servidor WSGI de Producción (Líneas 16-19): Expone `EXPOSE 80` y define `CMD ["gunicorn", "--bind", "0.0.0.0:80", "--workers", "2", "--access-logfile", "-", "main:app"]`. Ejecuta 2 workers concurrentes enlazados a todas las interfaces para atender peticiones enrutadas por el proxy de App Service.

3. Microservicio Flask de Procesamiento: api/main.py

Desacoplamiento de Configuración (Líneas 14-20): Extrae variables operativas mediante `os.getenv()` con valores de contingencia (`STORAGE_PATH="/home/processed"`, `MAX_DOCUMENT_SIZE_MB=50`, `PROCESSING_TIMEOUT_SECONDS=30`), permitiendo modificar el comportamiento desde Application Settings sin recompilar la imagen.

Resiliencia de Almacenamiento (Líneas 30-37): La función `ensure_storage_directory()` valida y recrea la ruta persistente `STORAGE_PATH` ante caídas o reinicios del nodo host.

Contratos de Endpoints REST:

- `GET /health`: Endpoint ligero que retorna `{"status": "healthy"}` para los sondeos de disponibilidad (Liveness Probe) de App Service.
- `POST /process`: Genera un `uuid4()`, evalúa el límite de tamaño, calcula métricas de caracteres/palabras, extrae entidades mock con índices de confianza, genera análisis de sentimiento y escribe el resultado en `STORAGE_PATH/<doc_id>.json`.
- `GET /documents` & `GET /documents/<id>`: Inspecciona el volumen montado y expone el catálogo histórico de documentos analizados.

Preguntas Oficiales de Examen de Certificación

Knowledge check oficial de Microsoft Learn y distribución de habilidades evaluadas en el Examen AI-200

DISTRIBUCIÓN DE HABILIDADES EVALUADAS EN EL EXAMEN OFICIAL AI-200 (GUÍA DE ESTUDIO MICROSOFT LEARN)

AI-200 Study Guide

1. Contenedores Azure — 20% – 25%	2. Datos de IA en Azure — 25% – 30%	3. Conexión de Servicios — 20% – 25%	4. Seguridad y Monitoreo — 20% – 25%
★ **Evaluado en Lab 2:** App Service Linux, Managed Identity, AcrPull, persistencia /home y Kudu.	Cosmos DB NoSQL, PostgreSQL con `pgvector`, caché semántica y similitud en Managed Redis.	Mensajería con Service Bus y Event Grid, microservicios dirigidos por eventos en Azure Functions.	Identidades Entra ID, Key Vault, App Configuration, observabilidad OpenTelemetry y consultas KQL.

Pregunta 1: Autenticación Zero-Trust para Extracción de Contenedores en App Service

Managed Identity & RBAC

Una empresa debe desplegar un contenedor de inferencia en Azure App Service extrayendo la imagen desde un Azure Container Registry privado. La política de seguridad corporativa prohíbe terminantemente almacenar contraseñas estáticas en las variables de aplicación y exige el principio de privilegios mínimos. ¿Qué combinación de acciones cumple este requisito?

A) Habilitar la cuenta de administrador en ACR y configurar `DOCKER_REGISTRY_SERVER_PASSWORD` en la Web App.
B) Habilitar System-Assigned Managed Identity en la Web App, asignar el rol `AcrPull` en el ACR y configurar `acrUseManagedIdentityCreds=true`.
C) Asignar el rol `Contributor` al App Service Plan sobre la suscripción de Azure.

✓ **Respuesta Correcta: B) Habilitar System-Assigned Managed Identity, rol AcrPull en ACR y acrUseManagedIdentityCreds=true**

Justificación Oficial: Managed Identity elimina completamente el uso de credenciales estáticas. Conceder el rol RBAC `AcrPull` en el ámbito del ACR otorga permisos exclusivos de lectura para extraer capas de imagen (privilegio mínimo). Establecer `acrUseManagedIdentityCreds=true` instruye al motor Kudu a solicitar tokens a Entra ID automáticamente.

Pregunta 2: Persistencia de Datos entre Reinicios en Web App for Containers

Almacenamiento Persistente

Una API en contenedor guarda archivos PDF procesados en la ruta `/home/processed`. Tras un reinicio automático por mantenimiento de plataforma, los archivos generados desaparecen. ¿Qué ajuste de configuración resuelve el problema?

A) Configurar la variable de aplicación `WEBSITES_ENABLE_APP_SERVICE_STORAGE=true` en la Web App.
B) Escalar el App Service Plan a un nivel P3v3 con más memoria RAM.
C) Modificar el Dockerfile para cambiar la ruta de trabajo de `/home/processed` a `/tmp/processed`.

✓ **Respuesta Correcta: A) Configurar WEBSITES_ENABLE_APP_SERVICE_STORAGE=true en la Web App**

Justificación Oficial: En Azure App Service para Linux, el almacenamiento del contenedor es efímero por defecto. La variable `WEBSITES_ENABLE_APP_SERVICE_STORAGE=true` monta un volumen CIFS respaldado por Azure Storage en el directorio `/home`, garantizando que los archivos persistan ante reinicios, despliegues y escalado horizontal.

Pregunta 3: Mitigación de Errores de Timeout en el Arranque del Contenedor

Startup Probe Tuning

Un contenedor de IA tarda 6 minutos en descargar pesos de modelos y cargar librerías en memoria durante el inicio. Al desplegarlo en App Service, la plataforma aborta el proceso indicando: *"Container did not respond to HTTP ping on port 80"* tras 230 segundos. ¿Qué configuración soluciona el fallo?

A) Aumentar el límite de espera agregando la Application Setting `WEBSITES_CONTAINER_START_TIME_LIMIT=600`.
B) Cambiar el protocolo de la aplicación de HTTP a gRPC.
C) Forzar el puerto en el Dockerfile usando `EXPOSE 443`.

✓ **Respuesta Correcta: A) Agregar la Application Setting WEBSITES_CONTAINER_START_TIME_LIMIT=600**

Justificación Oficial: El timeout de arranque por defecto en App Service es de 230 segundos. La variable `WEBSITES_CONTAINER_START_TIME_LIMIT` permite extender este margen hasta un máximo de 1800 segundos (30 minutos), permitiendo que contenedores con inicializaciones prolongadas completen su arranque sin ser terminados.

Pregunta 4: Restricciones de Cómputo para Contenedores Personalizados en App Service

SKU & Linux Requirements

Un desarrollador intenta crear una Web App para contenedores usando un App Service Plan en el SKU Free (F1) y la CLI rechaza el aprovisionamiento. ¿Cuál es el requisito arquitectónico de Azure que causa este error?

A) Los contenedores personalizados en Linux requieren un plan de nivel dedicado (Basic B1 o superior / Premium).
B) El SKU Free solo admite contenedores compilados en arquitecturas ARM64.
C) Los contenedores requieren obligatoriamente una red virtual con ExpressRoute configurado.

✓ **Respuesta Correcta: A) Los contenedores en Linux requieren un plan dedicado (Basic B1 o superior / Premium)**

Justificación Oficial: La funcionalidad de Web App for Containers en Linux no está soportada en los niveles compartidos (Free F1 o Shared D1). Requiere cómputo dedicado con máquinas virtuales dedicadas (SKU Basic B1 en adelante o Premium v2/v3).

Pregunta 5: Transmisión y Captura en Vivo de Registros de Contenedor

Observabilidad & Logging

Un equipo de operaciones necesita depurar en tiempo real las excepciones no capturadas emitidas por una aplicación Flask en contenedor. ¿Qué comando de Azure CLI transmite inmediatamente la salida estándar (stdout/stderr) a la consola local?

A) `az webapp log tail --name $APP_NAME --resource-group $RESOURCE_GROUP`
B) `az webapp deployment source show --name $APP_NAME`
C) `az monitor metrics list --resource $APP_NAME`

✓ **Respuesta Correcta: A) az webapp log tail --name $APP_NAME --resource-group $RESOURCE_GROUP**

Justificación Oficial: `az webapp log tail` abre un túnel de transmisión en tiempo real contra el endpoint SCM/Kudu de la aplicación, mostrando la salida estándar (stdout y stderr) de los workers de Gunicorn y del motor de Docker de forma instantánea.

Glosario de Términos, Conceptos Clave y Siglas de Azure

Definiciones normativas de Azure App Service, diccionario de siglas y mapa curricular oficial

1. Conceptos Arquitectónicos Centrales de Azure App Service

APP SERVICE PLAN (ASP)

Unidad de cómputo físico/virtual que determina la región, el número de instancias, el SO (Linux/Windows), vCPUs y memoria RAM asignados a una o varias Web Apps.

WEB APP FOR CONTAINERS

Servicio PaaS administrado que ejecuta contenedores Docker/OCI en Linux con balanceo de carga integrado, certificados SSL gratuitos y escalado automático.

SYSTEM-ASSIGNED IDENTITY

Identidad administrada ligada al ciclo de vida del recurso en Microsoft Entra ID. Proporciona autenticación sin contraseñas para interactuar con ACR y Key Vault.

ALMACENAMIENTO PERSISTENTE /HOME

Volumen de red compartido (SMB/CIFS) respaldado por Azure Storage montado en `/home` cuando `WEBSITES_ENABLE_APP_SERVICE_STORAGE=true`.

KUDU / SCM ENGINE

Plano de control y motor de gestión de App Service accesible en `<app>.scm.azurewebsites.net`, encargado de pulls de imagen, WebSSH y telemetría de log.

CONTAINER LOGGING STREAM

Mecanismo de captura y retransmisión en tiempo real de los flujos `stdout` y `stderr` del contenedor Docker hacia Azure CLI y Azure Monitor.

2. Diccionario de Siglas y Acrónimos Técnicos del Módulo

Sigla	Significado y Rol en Azure
AAS	**Azure App Service:** Plataforma PaaS administrada para hospedar aplicaciones web y contenedores.
ASP	**App Service Plan:** Asignación de recursos de proceso (CPU, memoria, SKU y escala) para Web Apps.
ACR	**Azure Container Registry:** Registro privado administrado para almacenamiento y compilación OCI.
RBAC	**Role-Based Access Control:** Control de acceso basado en roles (`AcrPull`, `Contributor`).
MI / MSI	**Managed Identity / Managed Service Identity:** Identidades sin contraseña gestionadas por Entra ID.
OCI	**Open Container Initiative:** Estándar de la industria para formatos de imagen y runtimes de contenedores.
KUDU	**Kudu Engine:** Motor de despliegue, administración y diagnóstico asociado a cada Web App.
SCM	**Site Control Manager:** Endpoint administrativo aislado en `<app>.scm.azurewebsites.net`.
FQDN	**Fully Qualified Domain Name:** Nombre de dominio completo (ej. `<app>.azurewebsites.net`).

Sigla	Significado y Rol en Azure
TLS	**Transport Layer Security:** Protocolo criptográfico (TLS 1.3) que cifra el canal HTTPS de la Web App.
CIFS / SMB	**Common Internet File System:** Protocolo de red usado para montar `/home` desde Azure Storage.
IMDS	**Instance Metadata Service:** Endpoint local `169.254.169.254` para emisión de tokens Entra ID.
SKU	**Stock Keeping Unit:** Nivel de tarificación y capacidad de cómputo (Basic B1, Premium P0v3).
WSGI	**Web Server Gateway Interface:** Interfaz estándar entre servidores web (Gunicorn) y Python Flask.
REST	**Representational State Transfer:** Arquitectura de servicios web sin estado sobre protocolo HTTP.
CORS	**Cross-Origin Resource Sharing:** Mecanismo de seguridad para autorizar peticiones entre dominios.
ARM	**Azure Resource Manager:** Plano de control declarativo para gestionar infraestructura en Azure.
CVE	**Common Vulnerabilities and Exposures:** Identificador público estandarizado de fallos de seguridad.

3. Mapa Curricular y Rutas de Aprendizaje Oficiales de Microsoft Learn (Curso AI-200T00-A)

Curso Oficial Microsoft: AI-200T00-A: Desarrollo de soluciones en la nube de IA en Azure (5 días / 120 horas lectivas)

Certificación Asociada: Microsoft Certified: Azure AI Cloud Developer Associate (Examen AI-200)

FASE 1: CÓMPUTO & CONTENEDORES (DOM. 1)

- **Módulo 1:** Hospedaje de contenedores en Azure (ACR, ACR Tasks serverless, OCI).
- **Módulo 2:** Implementación en Azure App Service (Linux PaaS, Managed Identity). **[★ Este Lab]**
- **Módulo 3:** Implementación y escalado con KEDA en Azure Container Apps (ACA).

FASE 2: DATOS & EMBEDDINGS (DOM. 2)

- **Módulo 4:** Soluciones de IA con Azure Cosmos DB for NoSQL y vector search.
- **Módulo 5:** Bases de datos vectoriales con Azure Database for PostgreSQL (`pgvector`).
- **Módulo 6:** Optimización de latencia y caché semántica con Azure Managed Redis.

FASE 3: INTEGRACIÓN & DEVOPS (DOM. 3 Y 4)

- **Módulo 7:** Integración desacoplada con Azure Service Bus, Event Grid y Functions.
- **Módulo 8:** Gestión de secretos y feature flags en Azure Key Vault y App Configuration.
- **Módulo 9:** Observabilidad de extremo a extremo, OpenTelemetry y diagnóstico KQL.

Microsoft Learn Study Guide: https://learn.microsoft.com/es-es/credentials/certifications/resources/study-guides/ai-200

Microsoft Worldwide Learning (WWL) • Curso AI-200T00-A